Category:

Reversing

Name:

Fake C2

Message:

This binary file is a simplified emulation of a implant that communicates with a C2 server to execute arbitrary tasks. Analyze the file and extract the flag.

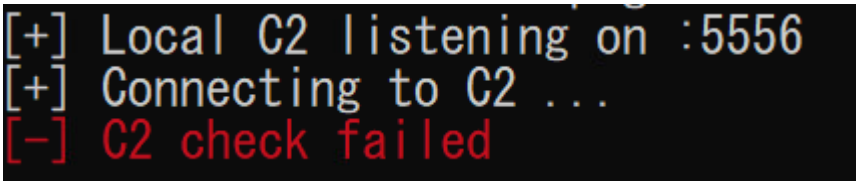
Extraction password is “csg2025”.

Warning: This file initiates simple communications that emulate a C2 server on localhost. However, **it does not contain anything harmful, nor does it make any external requests.**
When executed, the file only generates communications that are confined to localhost.
Additionally, when the file is executed, the server-side code embedded in the binary opens a specific port on localhost. If you notice any abnormal behavior, please check for potential port conflicts.

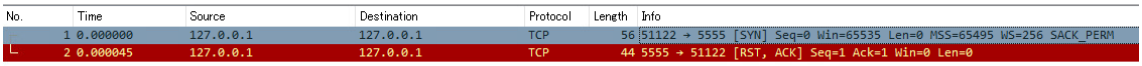
To proof your understanding, please create flag by “**CSG_FLAG{go_rev_startingpoint}**”

Instructions:

1. When the binary is first executed, the output shows that the **C2 is listening on localhost port 5556,** and that the communication with the C2 has failed.

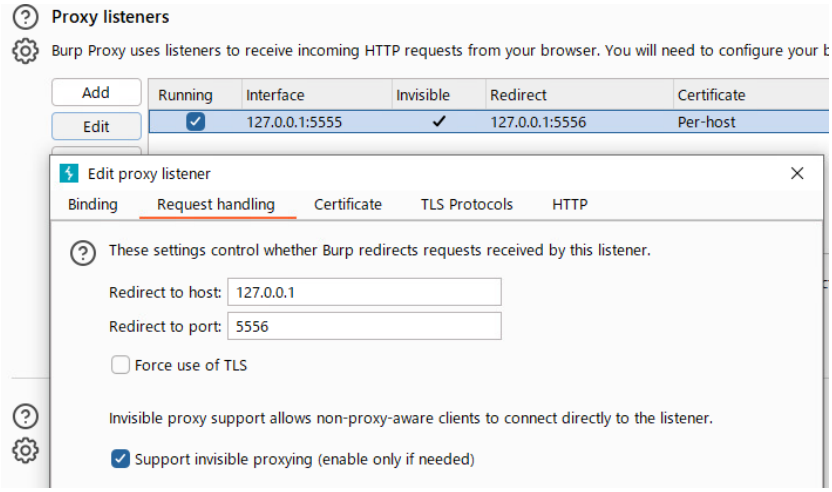


2. By examining the traffic with a packet capture tool such as Wireshark, you can observe that communication is occurring to **127.0.0.1:5555**

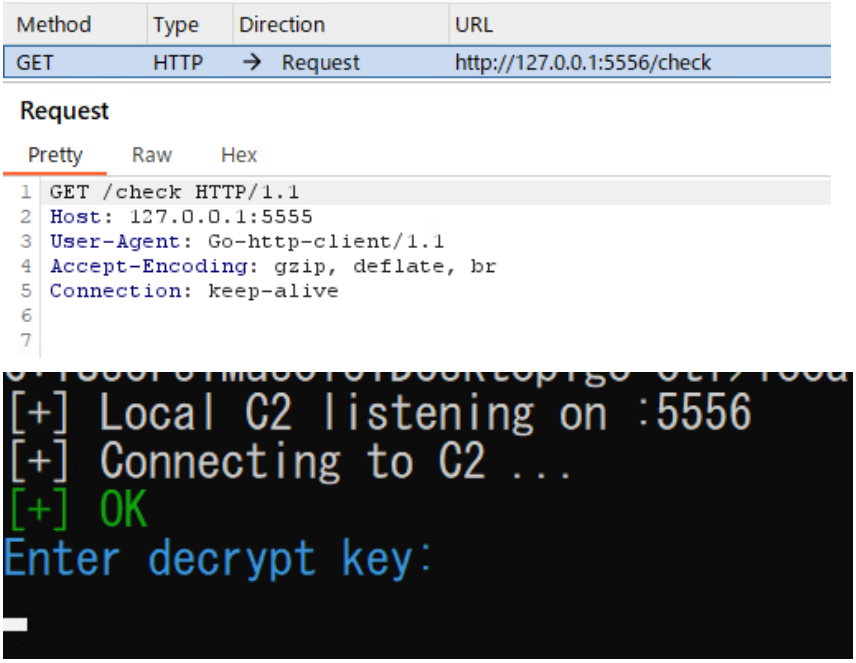


No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	56	51122 → 5555 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
2	0.000045	127.0.0.1	127.0.0.1	TCP	44	5555 → 51122 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

3. When the binary is executed, requests are sent to port 5555, but since the C2 is listening on port 5556, the communication fails. By binding the traffic with a tool such as Burp, the requests can be properly redirected to localhost:5556, allowing Burp to capture the communication.



4. From the Burp capture, it can be seen that a GET request is being sent to the endpoint **localhost/check**. Forwarding this request causes the console to prompt for the input of a decryption key.



5. By entering an arbitrary value and capturing the traffic, you can see that the **/post** endpoint receives the request containing the **decryptkey**. The response then returns what appears to be an encrypted value.
   To understand the details of this decryption process, reverse engineering is performed.

Request

Pretty Raw Hex

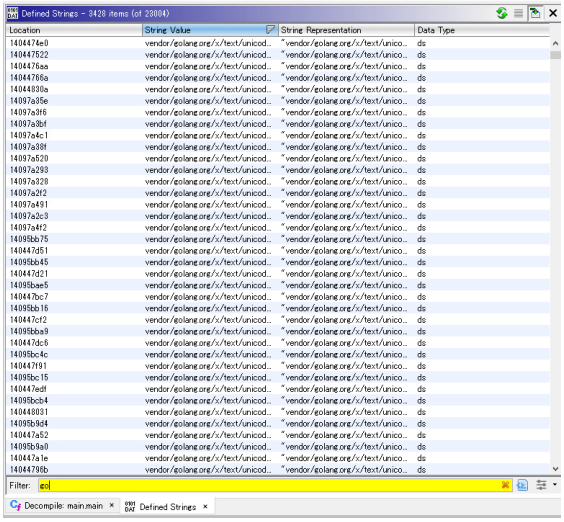
```
1 POST /post HTTP/1.1
2 Host: 127.0.0.1:5555
3 User-Agent: Go-http-client/1.1
4 Content-Length: 29
5 Content-Type: application/json
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8
9 {
    "decryptkey":"foobar"
  }
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json
3 Date: Mon, 15 Sep 2025 09:17:41 GMT
4 Content-Length: 68
5
6 □5Ár□ó%,
  □@³m`bÅw¸Áÿ°9*±~$
7 õ|+»3õÆ±□üL□«úbè/□Á▷□5«OÅÿ°%1[
    @å
```

6. Open the binary file with a reverse engineering tool such as Ghidra. In the case of Ghidra, by examining elements like **Defined Strings**, it can be inferred that **the file was written in Go.**



7. For Go binaries, it is common practice to check whether the C2 server URL is defined in **main.init** and to review the logic in **main.main**. Within main.main, there are functions such as **main.startServer** and **main.runClient**. Based on the previous execution results, it is evident that the binary contains both client and server code. **Since the decryption process occurs after a request is sent to the server and a response is returned, analyzing the server-side code may reveal the details of the decryption logic.**

Decompile: main.main - (localc2em.exe)

```
1
2 int __cdecl main.main(int _Argc,char **_Argv,char **_Env)
3
4 {
5   int iVar1;
6   longlong unaff_R14;
7
8   while (&stack0x00000000 < *(undefined **)(ulonglong *)(unaff_R14 + 0x10) ||
9          &stack0x00000000 == *(undefined **)(ulonglong *)(unaff_R14 + 0x10)) {
10    runtime.morestack_noctxt.abi0();
11  }
12  main.startServer(main.flag);
13  iVar1 = main.runClient();
14  return iVar1;
15}
```

8. Looking into the **main.startServer** function, several other functions are executed. While it can be inferred that one of them is responsible for the decryption process, it is not yet clear which one. Decompiling the **&main.startServer.func2** function reveals a call to the **XORKeyStream** function.



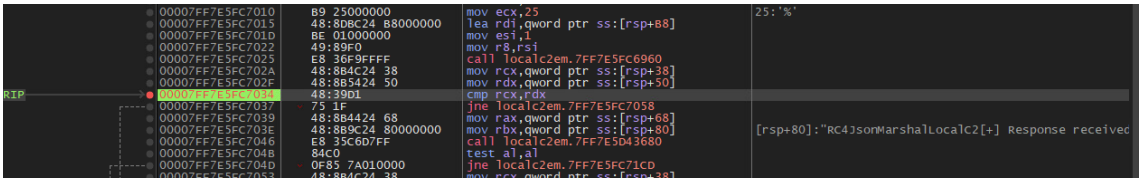


9. As noted below, XORKeyStream is used during both encryption and decryption processes. Furthermore, from the description, it can be seen that the first argument of

rc4.NewCipher is the key being passed. Therefore, it can be concluded that &main.startServer.func2 is the function responsible for the encryption process, and that **iVar7** receives the key data entered by the user.

RC4 (40-2048 bits) in Go | SSOJet

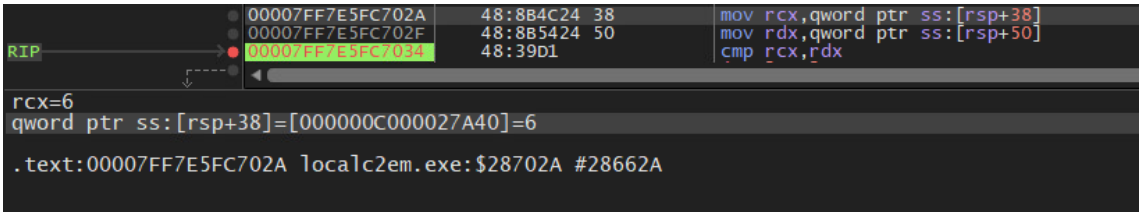
10. To further analyze this encryption function, open a debugger such as x64dbg and attach it to the process.
11. **Set a breakpoint at the conditional branch of the if statement**, located just before the decryption process with XORKeyStream as identified in Ghidra. The RVA visible in Ghidra is **0x287034**.
12. Execute the binary with breakpoints set and send a request.
    When a breakpoint is set at the if statement and a request is sent, execution halts at the **cmp rcx, rdx** instruction.

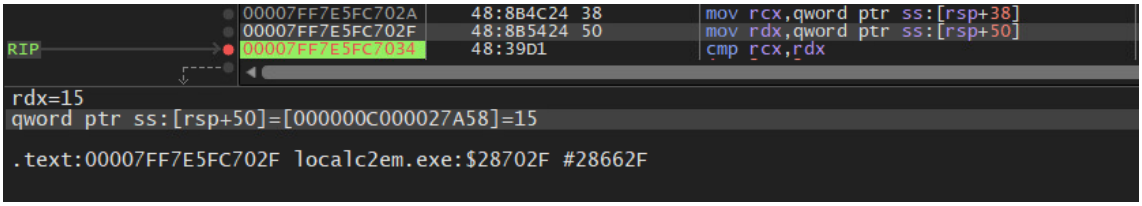


13. At this point, review the decompiled output of the decryption function in Ghidra once more. It becomes clear that the if statement enters the decryption process **when lVar7 matches lVar2. Since lVar2 is likely the original key information**, the condition is probably checking whether it matches the user-supplied key lVar7. Therefore, the value of lVar2 is inspected in the debugger.

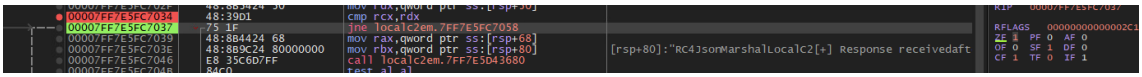
```
68    puStack_20 = &DAT_140305700;
69    github.com/fatih/color.colorPrint(0x25,&DAT_140305700,1);
70    if ((lVar7 == lVar2) && (cVar3 = runtime.memequal(), cVar3 != '\0')) {
71      uStack_48 = crypto/rc4.NewCipher(lVar7);
72      uStack_30 = runtime.makeslice(uVar1);
```

14. Before reaching the jne instruction, the values of rcx and rdx are compared. It can be confirmed that rcx holds the value 6, while rdx holds 15. In this case, the user input was the string **foobar, and the value 6 in rcx corresponds to the length of this input key.** On the other hand, rdx has the value 15, which is assumed to represent the expected key length. Since this 15 is in hexadecimal, **the actual expected key length is 21 characters.**



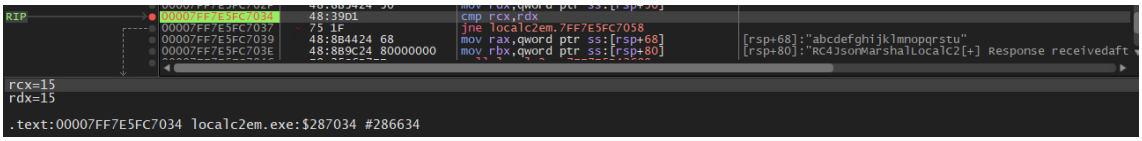


Note: rcx holds the length of the user-input key, while rdx contains the expected length of the original key. The program first compares these lengths, after which the jne instruction is executed. Therefore, if the flag for jne is not set, the Zero Flag (ZF) should be set to 1 before stepping through the execution.

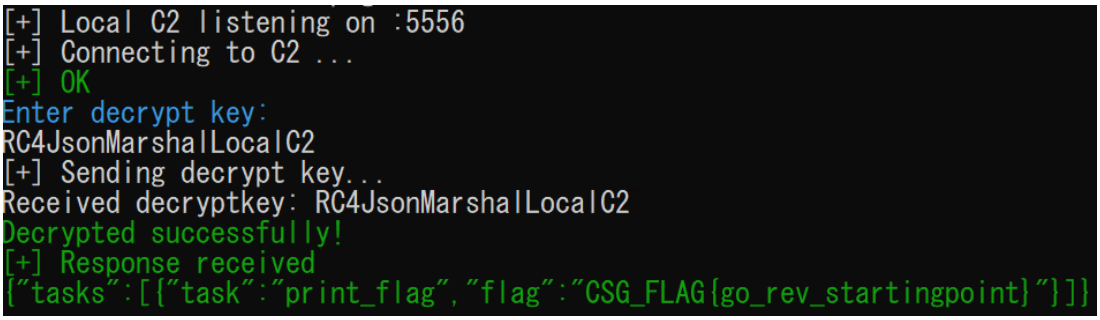


15. Therefore, a random string with **a length of 21 characters** is sent via POST. As a result, the sizes of rcx and rdx match during the cmp instruction, so the program does not jump at the jne.

    At this point, the user-input key can be observed at **rsp+68**, while the expected 21-byte key data can be found at **rsp+80**. Calculating the 21-character sequence from the RC4.... value reveals that the expected key is **RC4JsonMarshalLocalC2**.



16. Based on the information gathered so far, running the binary and entering **RC4JsonMarshalLocalC2** results in the flag being output.



Flag: **CSG_FLAG{go_rev_startingpoint}**